

Diffusion Models Exercise

Felex Hill

July 7, 2026

Setup and Notation

Recall the forward process

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, \beta_t I), \quad \alpha_t = 1 - \beta_t,$$

which, via the reparameterization trick, can be written as

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1}, \quad \epsilon_{t-1} \sim \mathcal{N}(0, I).$$

We write $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$.

1 Problem 1: Closed-Form Forward Process

Problem. Show that $q(x_t \mid x_0)$ is Gaussian, and that

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \bar{\epsilon}, \quad \bar{\epsilon} \sim \mathcal{N}(0, I).$$

Proof

We proceed by induction on t , repeatedly unrolling the one-step recursion and using the fact that a weighted sum of independent Gaussians is again Gaussian, with variances adding.

Base case ($t = 1$). By definition,

$$x_1 = \sqrt{\alpha_1} x_0 + \sqrt{1 - \alpha_1} \epsilon_0,$$

and since $\bar{\alpha}_1 = \alpha_1$, this is exactly the claimed form with $\bar{\epsilon} = \epsilon_0$.

Inductive step. Suppose the claim holds at step $t - 1$, i.e.

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} x_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \bar{\epsilon}_{t-1}, \quad \bar{\epsilon}_{t-1} \sim \mathcal{N}(0, I),$$

where $\bar{\epsilon}_{t-1}$ is independent of x_0 . Substituting into the one-step update,

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1} = \sqrt{\alpha_t} \left(\sqrt{\bar{\alpha}_{t-1}} x_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \bar{\epsilon}_{t-1} \right) + \sqrt{1 - \alpha_t} \epsilon_{t-1}.$$

Collecting the x_0 term and the two noise terms,

$$x_t = \underbrace{\sqrt{\alpha_t \bar{\alpha}_{t-1}}}_{=\sqrt{\bar{\alpha}_t}} x_0 + \underbrace{\sqrt{\alpha_t (1 - \bar{\alpha}_{t-1})} \bar{\epsilon}_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1}}_{\text{sum of two independent Gaussians}}.$$

Since $\bar{\epsilon}_{t-1}$ and ϵ_{t-1} are independent standard Gaussians, their weighted sum is itself Gaussian with mean 0 and variance equal to the sum of the two squared coefficients:

$$\alpha_t(1 - \bar{\alpha}_{t-1}) + (1 - \alpha_t) = \alpha_t - \alpha_t\bar{\alpha}_{t-1} + 1 - \alpha_t = 1 - \alpha_t\bar{\alpha}_{t-1} = 1 - \bar{\alpha}_t.$$

Hence the combined noise term is distributed as $\mathcal{N}(0, (1 - \bar{\alpha}_t)I)$, which we may write as $\sqrt{1 - \bar{\alpha}_t} \bar{\epsilon}$ for a single $\bar{\epsilon} \sim \mathcal{N}(0, I)$. Therefore

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \bar{\epsilon},$$

which completes the induction. Since x_t is an affine function of x_0 plus Gaussian noise, $q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$. ■

2 Problem 2: Implementing the Forward Process

Problem. Fill in the blank in the `q_sample` function in `diffusion_exercises.py` to implement the closed-form sampling of x_t given x_0 .

Directly implements the result of Problem 1, $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$, which lets us draw a noised sample x_t straight from x_0 at any timestep t without iterating through the intermediate steps.

```

1 def q_sample(x_0, t, noise=None):
2     if noise is None:
3         noise = torch.randn_like(x_0)
4
5     # Get the \bar{\alpha}_t values for the given timesteps t
6     alphas_t = get_alphas_cumprod_at_t(t)
7
8     # YOUR CODE HERE: calculate x_t
9     # Formula: x_t = sqrt(alpha_bar_t) * x_0 + sqrt(1 - alpha_bar_t) * noise
10    noisy_x = torch.sqrt(alphas_t) * x_0 + torch.sqrt(1 - alphas_t) * noise
11
12    return noisy_x

```

3 Problem 3: Reverse Process Posterior

Problem. Derive the mean and variance of $q(x_{t-1} | x_t, x_0) = \mathcal{N}(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$, showing

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t.$$

Proof

By Bayes' theorem,

$$q(x_{t-1} | x_t, x_0) \propto q(x_t | x_{t-1}) q(x_{t-1} | x_0),$$

where both factors on the right are Gaussian in x_{t-1} :

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_{t-1}, \beta_t I), \quad q(x_{t-1} | x_0) = \mathcal{N}(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}} x_0, (1 - \bar{\alpha}_{t-1}) I).$$

Working in the log domain (dropping additive constants that do not depend on x_{t-1}),

$$\log q(x_{t-1} | x_t, x_0) = -\frac{1}{2} \left(\frac{(x_t - \sqrt{\bar{\alpha}_t} x_{t-1})^2}{\beta_t} + \frac{(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}} x_0)^2}{1 - \bar{\alpha}_{t-1}} \right) + C.$$

Expanding both squares and collecting terms by power of x_{t-1} :

$$= -\frac{1}{2} \left[\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} \right) x_{t-1}^2 - 2 \left(\frac{\sqrt{\alpha_t} x_t}{\beta_t} + \frac{\sqrt{\bar{\alpha}_{t-1}} x_0}{1 - \bar{\alpha}_{t-1}} \right) x_{t-1} \right] + C'.$$

This is quadratic in x_{t-1} , so $q(x_{t-1} \mid x_t, x_0)$ is Gaussian. Matching against the general form $-\frac{1}{2\tilde{\beta}_t} (x_{t-1}^2 - 2\tilde{\mu}_t x_{t-1})$, we read off the precision (inverse variance) and the mean.

Variance. The coefficient of x_{t-1}^2 gives

$$\frac{1}{\tilde{\beta}_t} = \frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} = \frac{\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t}{\beta_t(1 - \bar{\alpha}_{t-1})}.$$

The numerator simplifies using $\beta_t = 1 - \alpha_t$ and $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$:

$$\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t = \alpha_t - \alpha_t \bar{\alpha}_{t-1} + 1 - \alpha_t = 1 - \bar{\alpha}_t.$$

Hence

$$\frac{1}{\tilde{\beta}_t} = \frac{1 - \bar{\alpha}_t}{\beta_t(1 - \bar{\alpha}_{t-1})} \implies \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t.$$

Mean. Completing the square, the mean is $\tilde{\beta}_t$ times the coefficient of the linear term:

$$\tilde{\mu}_t(x_t, x_0) = \tilde{\beta}_t \left(\frac{\sqrt{\alpha_t} x_t}{\beta_t} + \frac{\sqrt{\bar{\alpha}_{t-1}} x_0}{1 - \bar{\alpha}_{t-1}} \right).$$

Substituting $\tilde{\beta}_t = \frac{(1 - \bar{\alpha}_{t-1})\beta_t}{1 - \bar{\alpha}_t}$ and simplifying each term separately:

$$\begin{aligned} \tilde{\beta}_t \cdot \frac{\sqrt{\alpha_t} x_t}{\beta_t} &= \frac{(1 - \bar{\alpha}_{t-1})\beta_t}{1 - \bar{\alpha}_t} \cdot \frac{\sqrt{\alpha_t}}{\beta_t} x_t = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t, \\ \tilde{\beta}_t \cdot \frac{\sqrt{\bar{\alpha}_{t-1}} x_0}{1 - \bar{\alpha}_{t-1}} &= \frac{(1 - \bar{\alpha}_{t-1})\beta_t}{1 - \bar{\alpha}_t} \cdot \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} x_0 = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} x_0. \end{aligned}$$

Adding these two terms gives

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t,$$

as required. ■

4 Problem 4: The Neural Network's Goal

Problem. Show that the posterior mean $\tilde{\mu}_t(x_t, x_0)$ can equivalently be expressed in terms of x_t and the noise ϵ as

$$\tilde{\mu}_t(x_t, \epsilon) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right).$$

Proof

From Problem 1, $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$, so we can solve for x_0 in terms of x_t and ϵ :

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}.$$

Substitute this into the expression for $\tilde{\mu}_t(x_t, x_0)$ from Problem 3:

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \cdot \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}} + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t.$$

Since $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$, we have $\frac{\sqrt{\bar{\alpha}_{t-1}}}{\sqrt{\bar{\alpha}_t}} = \frac{1}{\sqrt{\alpha_t}}$, so the first term becomes

$$\frac{1}{\sqrt{\alpha_t}} \cdot \frac{\beta_t}{1 - \bar{\alpha}_t} (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon) = \frac{\beta_t}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)} x_t - \frac{\beta_t}{\sqrt{\alpha_t}\sqrt{1 - \bar{\alpha}_t}} \epsilon.$$

Thus

$$\tilde{\mu}_t = \left[\frac{\beta_t}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)} + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \right] x_t - \frac{\beta_t}{\sqrt{\alpha_t}\sqrt{1 - \bar{\alpha}_t}} \epsilon.$$

Combine the two coefficients of x_t over the common denominator $\sqrt{\alpha_t}(1 - \bar{\alpha}_t)$:

$$\frac{\beta_t + \alpha_t(1 - \bar{\alpha}_{t-1})}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)}.$$

The numerator simplifies using $\beta_t = 1 - \alpha_t$ and $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$:

$$\beta_t + \alpha_t(1 - \bar{\alpha}_{t-1}) = (1 - \alpha_t) + \alpha_t - \alpha_t \bar{\alpha}_{t-1} = 1 - \bar{\alpha}_t.$$

So the coefficient of x_t reduces to

$$\frac{1 - \bar{\alpha}_t}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)} = \frac{1}{\sqrt{\alpha_t}}.$$

Putting it together,

$$\tilde{\mu}_t(x_t, \epsilon) = \frac{1}{\sqrt{\alpha_t}} x_t - \frac{\beta_t}{\sqrt{\alpha_t}\sqrt{1 - \bar{\alpha}_t}} \epsilon = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right),$$

as required. This shows that once a network $\epsilon_\theta(x_t, t)$ can predict the noise ϵ , the entire reverse process mean — and hence the full reverse process — is recovered. ■

5 Problem 5: The Training Loop

Problem. Fill in the blanks in the code in `diffusion_exercises.py` to complete the training loop.

At each step, we sample noise $\epsilon \sim \mathcal{N}(0, I)$, build the noisy sample x_t via `q_sample` (Problem 2), and train the network $\epsilon_\theta(x_t, t)$ to predict ϵ using the simplified MSE loss $L_{\text{simple}} = \mathbb{E} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$.

```

1 # ... (inside the training loop) ...
2 # 3. Sample random noise from standard Gaussian distribution
3 # YOUR CODE HERE
4 noise = torch.randn_like(x_0)
5

```

```

6 # 4. Create the noisy sample x_t using the q_sample function
7 # YOUR CODE HERE
8 x_t = q_sample(x_0, t, noise=noise)
9
10 # ... (model prediction) ...
11 predicted_noise = model(x_t, t)
12
13 # 6. Calculate the loss with the MSE (mean square error) function
14 # YOUR CODE HERE
15 loss = torch.mean((predicted_noise - noise) ** 2)

```

6 Problem 6: The Sampling Loop

Problem. Fill in the blanks in the `p_sample_loop` function in `diffusion_exercises.py`. This involves using the trained model ϵ_θ to calculate the mean of the reverse step and then drawing a sample.

Implements the reverse step $x_{t-1} \sim \mathcal{N}(\tilde{\mu}_t(x_t, \epsilon_\theta), \tilde{\beta}_t I)$, using the mean formula from Problem 4 (with the true noise ϵ replaced by the network prediction $\epsilon_\theta(x_t, t)$) and the variance formula from Problem 3.

```

1 # The mean of the distribution for x_{t-1}
2 # YOUR CODE HERE
3 mean = (1 / torch.sqrt(alpha_t)) * (
4     x - ((1 - alpha_t) / torch.sqrt(1 - alpha_t_cumprod)) * predicted_noise
5 )
6
7 # Get the variance and add noise
8 if t_val > 0:
9     alpha_t_cumprod_prev = alphas_cumprod[t_val - 1]
10
11     # The variance of the distribution for x_{t-1}
12     # YOUR CODE HERE
13     posterior_variance = beta_t * (1 - alpha_t_cumprod_prev) / (1 -
14         alpha_t_cumprod)
15
16     noise = torch.randn_like(x) * torch.sqrt(posterior_variance)
17 else: # No noise at the last step
18     noise = 0
19
20 x = mean + noise

```

7 Experimental Results

The model was trained for 10,000 epochs ($T = 1000$ diffusion steps, linear β_t schedule from 10^{-4} to 0.02) on a two-moons dataset. Training loss decreased from 0.972 at epoch 0 to approximately 0.22–0.24 by epoch 10,000, at which point it had plateaued (Figure 1).

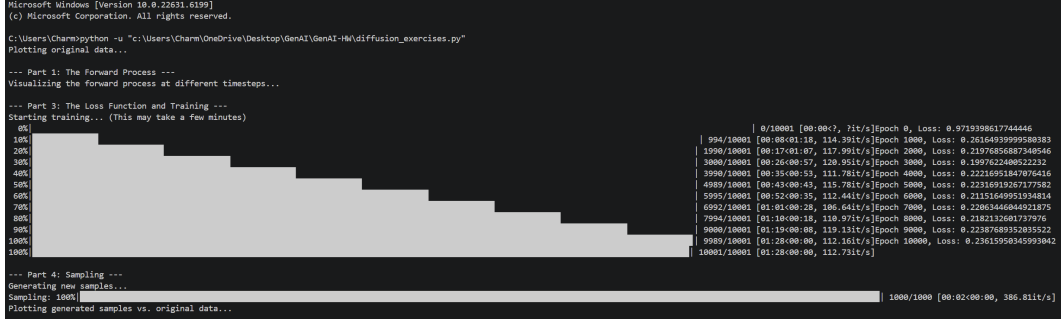


Figure 1: Training and sampling log. Loss drops sharply within the first 1000 epochs and then fluctuates around 0.21–0.24 for the remainder of training.

Figure 2 shows the forward process progressively destroying the structure of the data as t increases, converging toward isotropic Gaussian noise by $t = 500$.

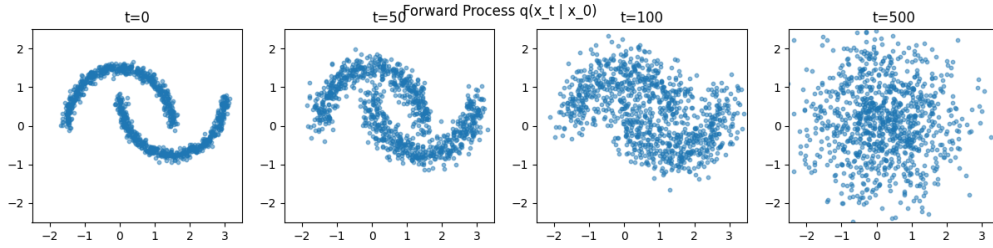
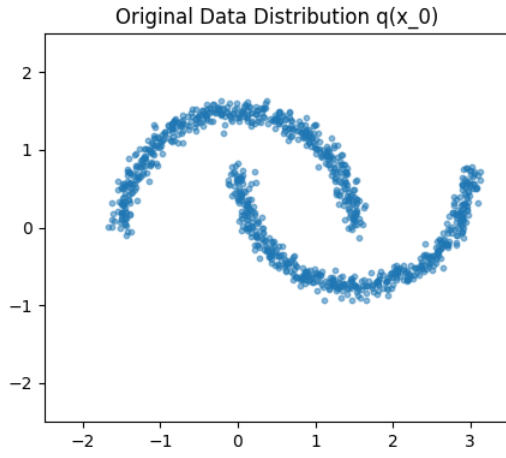
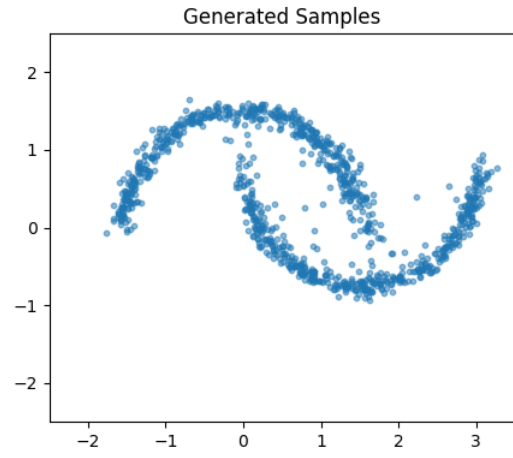


Figure 2: Forward process $q(x_t | x_0)$ at $t = 0, 50, 100, 500$. The two-moons structure is progressively destroyed as noise accumulates.

Finally, Figure 3 compares the original data distribution to samples generated by running the learned reverse process from pure Gaussian noise $x_T \sim \mathcal{N}(0, I)$. The generated samples closely recover the two-moons structure, confirming that the trained noise predictor ϵ_θ has learned a reverse process consistent with the derivations in Problems 1, 3, and 4.



(a) Original data $q(x_0)$



(b) Samples generated by the trained model

Figure 3: Comparison between the original data distribution and samples generated by reversing the diffusion process starting from pure noise.